



An Optimal Algorithm for Computing $(\leq k)$ -Levels, with Applications to Separation and Transversal Problems*

Hazel Everett[†]

Jean-Marc Robert[‡]

Marc van Kreveld[§]

Abstract

This paper gives an optimal $O(n \log n + nk)$ time algorithm for constructing the levels $1, \dots, k$ in an arrangement of n lines in the plane. This algorithm is extended to compute these levels in an arrangement of n unbounded x -monotone polygonal convex chains, of which each pair intersects at most a constant number of times.

These algorithms can be used to solve the following separation and transversal problems. For a set of n blue points and a set of n red points, find a line that separates the two sets in such a way, that the sum, m , of the number of red points above the line and the number of blue points below the line is minimized. Such an optimal line can be found in $O(nm \log m + n \log n)$ time. For a set of n line segments in the plane, find a line that intersects the maximum number of the line segments. Such an optimal line can be found in $O(nm \log m + n \log n)$ time for vertical segments and in $O((nm \log m + n \log^2 n) \alpha(n))$ expected time for arbitrary line segments, where m denotes the number of line segments not intersected by the optimal line.

*This research is supported by NSERC and FCAR.

[†]Université du Québec à Montréal

[‡]Université du Québec à Chicoutimi

[§]McGill University

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.

9th Annual Computational Geometry, 5/93/CA, USA
© 1993 ACM 0-89791-583-6/93/0005/0038...\$1.50

1 Introduction

A k -level in an arrangement of n lines is the set of all points that lie above or on at least k lines, and below or on at least $n - k + 1$ lines. As a consequence, a k -level is a subset of the edges and vertices of the arrangement of the lines (see Figure 1). The concept of k -levels is strongly related to k -sets by the well-known duality transform. For a set S of n points in the plane, a subset P of S is called a k -set of S if and only if there exists a line l , such that all points of P lie above l , and all points in $S - P$ lie below l , or vice versa. The concepts of k -sets and k -levels have proved to be important in computational geometry. An intriguing open problem is to establish sharp bounds on the maximum complexity of k -sets.

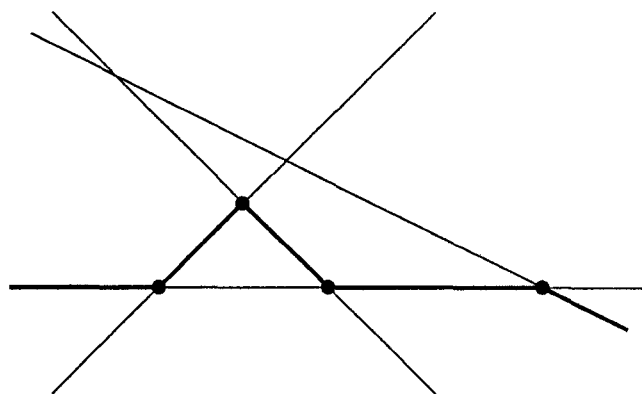


Figure 1: The upper 3-level (or lower 2-level) in an arrangement of four lines.

We are interested in the efficient construction of k -levels, and the construction of all i -levels for $1 \leq i \leq k$ (also called the $(\leq k)$ -levels). For the construction of a k -level, Edelsbrunner and

Welzl[10] developed an $O(f_k(n) \log^2 n)$ time algorithm, where $f_k(n)$ is the maximum complexity of the k -level. Cole, Sharir and Yap [6] improved their algorithm slightly if k is small; their algorithm runs in $O(n \log n + f_k(n) \log^2 k)$ time.

For the construction of $(\leq k)$ -levels, we first remark that the total complexity of these levels is $O(nk)$, and this is tight. As a consequence, Edelsbrunner and Welzl's result implies an $O(nk \log^2 n)$ algorithm to compute the $(\leq k)$ -levels. A better algorithm was given recently by Mulmuley[16]. He showed how to compute the $(\leq k)$ -levels in $O(nk \log(n/k))$ expected time. This paper presents the first optimal $O(n \log n + nk)$ time algorithm for this problem. It is easy to see that this bound is optimal by the tight bound on the complexity, and because the 1-level is dual to the upper convex hull.

We extend our results to the computation of k -levels and $(\leq k)$ -levels in arrangements of n unbounded x -monotone polygonal convex chains. If each pair of chains intersects at most twice, and N is the total complexity of the n chains (not of the arrangement), then the algorithm to compute the $(\leq k)$ -levels runs in $O((N/k + n) \log^2 n + N + nk)$ expected time, which is optimal if $k = \Omega(\log^2 n)$. The algorithm requires roughly the same amount of time if any two chains intersect at most a constant number of times. This improves a result of Sharir[20], who considered the case $N = O(n)$ and obtained an $O(nk \log n \log(n/k))$ time algorithm.

Our algorithms for computing the levels of an arrangement can be used to solve some separation and transversal problems. In Section 3, we consider the following problems. Let B be a set of n blue points and R a set of n red points. A *strong separator* for B and R is a line l such that $B \subset l^+$ and $R \subset l^-$ (or vice versa), where l^+ and l^- denote the closed half-planes above and below l , respectively. A strong separator can be computed in linear time, if one exists, by linear programming (see Megiddo[13]). A *weak separator* is a line l which minimizes the number of misclassified points, where any blue point below l and any red point above l is misclassified (see Figure 2). Houle[12] gave an $O(n^2)$ time algorithm for this problem. For a given k , Cole, Sharir and Yap[6] presented an $O(nk \log^2 n)$ time algorithm to com-

pute the separators misclassifying at most k points. We improve upon these results if the number of misclassified points is not too large. In particular, if m is the number of misclassified points, then our algorithm takes $O(nm \log m + n \log n)$ time. In this case, the value of m is not given as input, but it is determined by the algorithm. Notice that the algorithm is in a sense quality-sensitive, i.e., if there exists a good separating line for B and R , then it can be found fast. We extend the ideas to develop a similar algorithm for the weak separation of sets of red and blue polygons. In this case, we apply the $(\leq k)$ -level construction algorithm for sets of polygonal convex chains. Similar techniques can be used to separate one line segment from as many as possible amongst a family of n line segments with a line, in $O(n\sqrt{m} \log^2 n \log m)$ time, where m is the number of segments not separated from the one segment.

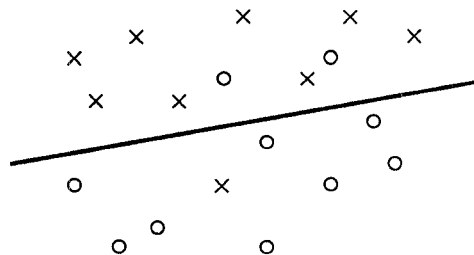


Figure 2: A weak separator for a set of blue points (crosses) and a set of red points (circles). Three points are misclassified.

In Section 4, we consider line transversal problems. Let S be a set of n line segments in the plane. A *line transversal* for S is a line which intersects all the line segments in S . The problem of computing a line transversal for a set of vertical line segments was considered by O'Rourke, who gave an $O(n \log n)$ time algorithm[17], which was improved later by linear programming. To find a line that intersects the maximum number of vertical line segments, we again give a quality-sensitive algorithm; it runs in $O(nm \log m + n \log n)$ time, where m is the number of segments not intersected by the optimal line. For arbitrary line segments, our algorithm takes $O((nm \log m + n \log^2 n) \alpha(n))$ expected

time, where $\alpha(n)$ is the extremely slowly growing functional inverse of Ackermann's function. This result may be compared to the $O(n \log n)$ time algorithm for a line transversal by Edelsbrunner et al.[9] and the $O(n^2)$ time algorithm to find a line intersecting the maximum number of segments by Edelsbrunner and Guibas[8].

Before we present our algorithms, we first state some results on k -sets and k -levels, and introduce some notation. The *lower* k -level in an arrangement $\mathcal{A}(H)$ of n lines is the set of points that lie above or on k lines of H and below or on $n - k + 1$ lines of H . The *upper* k -level is defined analogously, with above and below reversed. The number of points common to the upper k - and $(k + 1)$ -levels plus the number of points common to the lower k - and $(k + 1)$ -levels is precisely the number of k -sets in the set $\mathcal{D}(H)$ of n points dual to the lines of H . Let $f_k(H)$ denote the complexity of the upper or lower k -level in $\mathcal{A}(H)$, and let $f_k(n)$ denote the maximum complexity of the upper k -level or lower k -level in any arrangement of n lines. Let $g_k(n)$ denote the maximum number of k -sets in any set of n points. Furthermore, we denote by $F_k(n)$ and $G_k(n)$ the maximum combinatorial complexity of the $(\leq k)$ -levels and the maximum number of $(\leq k)$ -sets respectively. It is well-known that $f_k(n) \in \Theta(g_k(n) + g_{k+1}(n))$ which implies that $F_k(n) \in \Theta(G_{k+1}(n))$, if $1 \leq k \leq n/2$. Below we state the main results on the complexity of $f_k(n)$ and $F_k(n)$; for more details we refer to [7]:

Theorem 1 *For any $n \geq 1$ and $1 \leq k \leq n/2$:*

- $f_k(n) \in \Omega(n \log k)$ [7, 11],
- $f_k(n) \in O(n\sqrt{k}/\log^*(k+1))$ [19],
- $F_k(n) \in \Theta(nk)$ [2, 7].

For any arrangement of n unbounded x -monotone Jordan curves, of which each pair intersects at most s times (for a constant s), and $1 \leq k \leq n/2$, we have:

- $F_k(n) \in \Theta(k^2 \lambda_s(n/k))$ [20].

($\lambda_s(n)$ is the maximum length of a Davenport-Schinzel sequence with n symbols and of order s .)

$\lambda_s(n) \in O(n)$ if $s = 1$ or $s = 2$, $\lambda_3(n) \in \Theta(n\alpha(n))$, and for all constants $s \geq 3$, $\lambda_s(n)$ is slightly super-linear, see e.g. [1].)

2 Construction of the $(\leq k)$ -levels

In this section we give an algorithm that computes the upper $(\leq k)$ -levels in an arrangement of a set H of n lines optimally in $O(n \log n + nk)$ time. We may assume that $k < n/2$, otherwise we simply construct the full arrangement and extract the $(\leq k)$ -levels from it. The algorithm is based on an efficient deterministic algorithm of Cole, Sharir and Yap[6] for the construction of the k -level which takes $O(n \log n + (n + f_k(H)) \cdot \log^2 k)$ time. We give a short description of this algorithm; later we show how this leads to an algorithm that computes the upper $(\leq k)$ -levels optimally. For simplicity, we assume that no three lines intersect in one point.

We reformulate the problem in the dual space¹. Given a set $S = \mathcal{D}(H)$ of n points, find all lines that contain two points and have exactly $k - 1$ points below it, sorted by slope. Such a sequence of lines corresponds to the sequence of vertices of the upper k -level, sorted by x -coordinate. However, this sequence consists only of the vertices of the upper k -level that are also vertices of the upper $(k + 1)$ -level. Therefore, if $k > 1$, we should rerun the algorithm with $k - 1$. The second run gives all vertices of the upper k -level that are also vertices of the upper $(k - 1)$ -level in sorted order. Then we merge the two sequences to obtain all vertices of the upper k -level in sorted order.

The first step is to compute the convex layers of S . The convex hull of S is layer 1. For $i > 1$, the i -th layer is the convex hull of the points in S with the points in layers 1 to $i - 1$ removed (see Figure 3). Chazelle[3] showed how to solve this problem optimally in $O(n \log n)$ time. Since any line that intersects the k -th layer contains at least k points on each side, we can remove all layers deeper than k from consideration.

The algorithm basically is a rotational sweep. To get the sweep started, we compute a vertical

¹We use the duality transform as defined in [7].

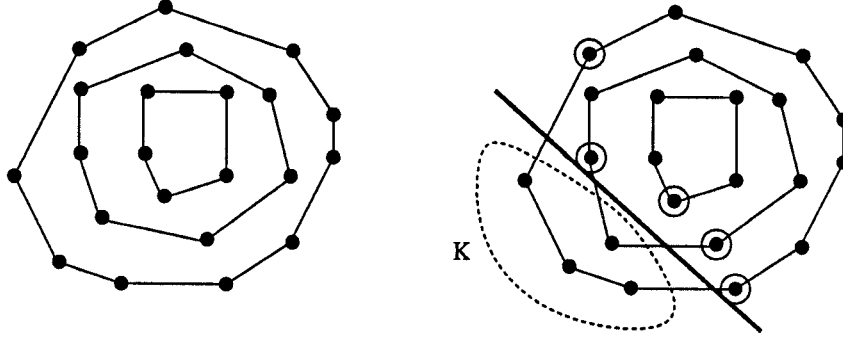


Figure 3: The convex layers (left) and the rotational sweep (right). The candidate points, C , are circled.

line that has k points to the left of it. This set of k points is the current k -set, denoted K . It is maintained in a dynamic convex hull maintenance structure, as developed by Overmars and van Leeuwen[18]. The structure has size $O(k)$, and insertions and deletions require $O(\log^2 k)$ time.

All points of the set are stored in circular lists that correspond to the at most k layers, which enable us to find the points adjacent to a given point in a given layer in constant time. Such an adjacent point in one layer is called a neighbor. Any point has two neighbors (except when the innermost layer consists of one or two points), which are referred to as its clockwise and counterclockwise neighbors. We maintain a set C of $O(k)$ candidates which represent the points that may be added to the set K in the next step of the rotational sweep. C consists of the following points (see Figure 3):

1. For any point $p \in K$ that is in layer i , its neighbor (or neighbors) in layer i is in C , provided that it is not in K .
2. For any layer i that has no point in K , the point closest to the rotating line is in C .

The set C of candidates is also maintained in a dynamic convex hull structure. A carefully implemented rotational sweep algorithm yields the following result:

Theorem 2 [6] *The k -level of a set H of n lines can be constructed in $O(n \log n + (f_k(H) + n) \cdot \log^2 k)$ time.*

Next we show how to compute the arrangement above the upper k -level. Once this arrangement is computed, we can extract the i -level for every $1 \leq i \leq k$ in time proportional to its size by a simple traversal.

To construct the full arrangement above the upper k -level, we first compute all lines, half-lines and segments that contribute to the $(\leq k)$ -level. Any line l lying strictly below the k -level is discarded. Any line l lying strictly above the k -level is kept. Otherwise, compute the rightmost and leftmost intersection points of a line l with the k -level. These two points partition l into two half-lines and a line segment. If either (or both) of the half-lines lie above the k -level, it is (they are) joined to the line segment. The resulting line, half-line or segment is kept. In total, at most n segments, half-lines and lines are chosen.

Lemma 1 *For any line l in a set H of n lines, the chosen segment, half-line or line does not contain a point below the upper $(2k - 1)$ -level.*

Proof: Let a and b be the leftmost and rightmost intersection points of l with the upper k -level of $\mathcal{A}(H)$. The half-line of l to the left of a either lies strictly above, or strictly below the k -level. We only choose it in the former case. For the half-line to the right of b , the argument is similar. Next, we consider the part of the line l between a and b . Suppose there is a point p between a and b which lies strictly below $2k - 1$ lines. Since a lies on the upper k -level, there must be at least k of these lines lying below or on a and intersecting l between a and b .

This implies that at least k lines lie strictly above b , contradicting the fact that b lies on the upper k -level. $\square$

To compute the arrangement above the upper k -level, we use the deterministic algorithm of Chazelle and Edelsbrunner[4] for the set of chosen segments, half-lines and lines (the randomized algorithms of Clarkson[5] or Mulmuley[14] lead to an expected time bound, but they work well in practice). They construct the arrangement of n line segments in the plane in $O(n \log n + A)$ time, where A is the size of the arrangement. Lemma 1 shows that we do not compute any edge or vertex below the upper $(2k - 1)$ -level. Therefore, the complexity of the arrangement that is constructed is $O(nk)$. To obtain the $(\leq k)$ -levels, we remove all parts of the arrangement that lie below the upper k -level in a straightforward way. Hence, we obtain the following result.

Theorem 3 *Given a set H of n lines in the plane, the $(\leq k)$ -levels can be constructed optimally in $O(n \log n + nk)$ time.*

Next we generalize to the problem of constructing levels in arrangements of unbounded x -monotone polygonal convex chains. Let H be a set of n such chains, and let the total size complexity of the chains be N . Furthermore, assume that each pair of chains intersects in at most s points, for a constant s . It has been shown by Sharir[20] that the total size complexity of the $(\leq k)$ -levels is $O(N + k^2 \lambda_s(n/k))$, see Theorem 1.

Not all ideas used to construct the $(\leq k)$ -levels in an arrangement of lines extend to this case. However, we can construct a k -level in $\mathcal{A}(H)$ in $O((N + b_k(n)) \log^2 n)$ time, where $b_k(n)$ is the number of intersection points of two different chains of H in the k -level by adapting the algorithm of Edelsbrunner and Welzl[10]. As the following lemma shows, it turns out to be better to construct the k' -level for a random k' between k and $2k$, than the k -level itself.

Lemma 2 *With probability at least $1/2$, the complexity of the k' -level for a randomly chosen k' that lies between k and $2k$ is $O(N/k + k \lambda_s(n/k))$.*

Proof: Suppose there are more than $k/2$ levels in the given range, for which the complexity is greater than $c \cdot (N/k + k \lambda_s(n/k))$, for a sufficiently large constant c . Then the complexity of all $(\leq 2k)$ -levels is at least $c \cdot (N/k + k \lambda_s(n/k)) \cdot k/2 = c \cdot N/2 + c \cdot k^2 \lambda_s(n/k)/2$, which contradicts the fact that this complexity is $O(N + k^2 \lambda_s(n/k))$, if c is chosen large enough. $\square$

Hence, if we choose a random value for k' in the specified range, the complexity of the k' -level will be $O(N/k + k \lambda_s(n/k))$ with probability at least $1/2$. We compute the k' -level, but abort the computation as soon as the complexity of the k' -level is more than $c \cdot (N/k + k \lambda_s(n/k))$. In this case, we choose a new random k' and restart. With high probability, we only have to restart a constant number of times. Hence, the algorithm takes $O((N/k + k \lambda_s(n/k)) \log^2 n)$ expected time. To compute the arrangement above the upper k' -level, we choose all edges from the k' -level with their extensions above the k' -level, and also the $O(n)$ full polygonal chains above the k' -level. To construct arrangement of these polygonal convex chains, we apply Mulmuley's randomized algorithm[15], which takes $O(n \log n + A)$ time, where A is the size of the arrangement. Hence, we obtain:

Theorem 4 *Given a set H of n unbounded x -monotone polygonal convex chains of total complexity N , such that each pair intersects at most s times, the $(\leq k)$ -levels can be constructed in $O((N/k + k \lambda_s(n/k)) \log^2 n + N + k^2 \lambda_s(n/k))$ expected time. (If $s \leq 2$, then the bound is $O((N/k + n) \log^2 n + N + nk)$; if $N \in O(n)$ and $s \leq 2$, then the bound is $O(n \log^2 n + nk)$.)*

3 The weak separation problem

Let $B = \{b_1, \dots, b_n\}$ be a set of n blue points, and let $R = \{r_1, \dots, r_n\}$ be a set of n red points. A line l is a strong separator of B and R if all points of B lie in the one closed half-plane defined by l , and all points of R lie in the other closed half-plane defined by l . Note that both red and blue points on l are allowed. The problem of finding a strong separator can be transformed to a 2-dimensional

linear programming problem, and hence, can be solved in linear time (Megiddo[13]).

A line l is a weak separator if the minimum of $|B \setminus l^+| + |R \setminus l^-|$ and $|B \setminus l^-| + |R \setminus l^+|$ is minimized. Intuitively, a weak separator is a line that separates two sets of points, and minimizes the number of misclassified points. Houle[12] presented an algorithm for the weak separator problem which runs in $O(n^2)$ time. His algorithm is based on a topological line sweep on the arrangements of lines dual to the sets of points B and R . Our algorithm improves upon this result if the number of misclassified points is sublinear. More precisely, the algorithm requires $O(nm \log m + n \log n)$ time, where m is the number of misclassified points. For simplicity, we only look for weak separators that have most blue points above or on the separating line, and most red points below or on it. This restriction can easily be removed. Finally, we may assume that $0 < m < n/2$, otherwise we can use the known linear time or the $O(n^2)$ time algorithm.

We first dualize the sets B and R to sets $\mathcal{D}(B)$ and $\mathcal{D}(R)$ of blue and red lines, respectively. A line l in primal space with m_b blue points below it and m_r red points above it (the misclassified points) dualizes to a point $\mathcal{D}(l)$ that has m_b blue lines above it and m_r red lines below it. This follows immediately from the properties of dualization. Clearly, the boundary of the locus of points in dual space with at most m_b blue lines above it is the upper $(m_b + 1)$ -level of the arrangement $\mathcal{A}(\mathcal{D}(B))$. Similarly, the boundary of the locus of points in dual space with at most m_r red lines below it is the lower $(m_r + 1)$ -level of the arrangement $\mathcal{A}(\mathcal{D}(R))$. Hence, if the upper $(m_b + 1)$ -level of $\mathcal{A}(\mathcal{D}(B))$ intersects (or lies completely below) the lower $(m_r + 1)$ -level of $\mathcal{A}(\mathcal{D}(R))$, then there exists a separating line that misclassifies at most $m_r + m_b$ points.

Lemma 3 *In $O(n\sqrt{t} \log^2 t + n \log n)$ time, one can determine whether there exists a separating line that misclassifies at most t blue points and at most t red points. Any subsequent question with a new value of t requires $O(n\sqrt{t} \log^2 t)$ time.*

This follows from the algorithm used in Theorem 2 and the fact that the upper and lower t -levels can

be intersected in time proportional to the sum of their lengths. The latter statement in the lemma is true because the convex layers of the two sets of points do not have to be recomputed.

The algorithm to compute a weak separator of B and R first approximates the number of points m misclassified by a weak separator by determining the smallest integer j such that there exists a separator misclassifying at most 2^j points of each color. Start with $j = 0$ and determine with the algorithm of Lemma 3 whether a separator exists that misclassifies at most 2^j points of each color. If so, then return j , otherwise, increment j and repeat.

At this point it is known that a weak separator misclassifying at least 2^{j-1} points and at most 2^{j+1} points exists. Hence, $m \in \Theta(2^j)$. Given sets B and R and an integer $2^{j-1} < t \leq 2^{j+1}$, we determine whether there exists a separator that misclassifies at most t points in total as follows. Using the algorithm of the previous section, we compute the upper $(\leq 2^{j+1} + 1)$ -levels in the arrangement $\mathcal{A}(\mathcal{D}(B))$ and the lower $(\leq 2^{j+1} + 1)$ -levels in the arrangement $\mathcal{A}(\mathcal{D}(R))$.

For i ranging from 0 to t , decide if the blue upper $(i + 1)$ -level intersects the red lower $(t - i + 1)$ -level by traversing them simultaneously from left to right. The time for the traversal is clearly linear in the sum of the complexities of the $(i + 1)$ -level and the $(t - i + 1)$ -level. Since the complexity of the $(\leq t)$ -levels is in $O(nt)$, we conclude:

Lemma 4 *Given the blue upper $(\leq t)$ -levels and the red lower $(\leq t)$ -levels, one can determine whether there exists a separating line that misclassifies at most t points in $O(nt)$ time.*

We proceed as follows to compute the exact value of the minimum number of misclassified points m . Perform a binary search for m between the values 2^{j-1} and 2^{j+1} . If mid is the current middle value of the known range for m , we decide with the algorithm of Lemma 4 whether $m > mid$ or $m \leq mid$. After $O(\log 2^{j+1}) \subseteq O(\log m)$ steps the exact value of m is found. A weak separator is the line dual to a point where the upper $(i + 1)$ -level of $\mathcal{A}(\mathcal{D}(B))$ intersects the lower $(m - i + 1)$ -level of $\mathcal{A}(\mathcal{D}(R))$.

To prove a time bound on the algorithm for finding a weak separator misclassifying m points, we

first establish how efficiently we can compute the value of j . By using the algorithm of Lemma 3, j is computed in $O(n \log n + \sum_{i=0}^j n \sqrt{2^i} \log^2(2^i)) = O(n \log n + n \sqrt{m} \log^2 m)$ time. Furthermore, each of the $O(\log m)$ iterations of the binary search takes time $O(nm)$ time by using the algorithm of Lemma 4. Hence, we obtain the following result:

Theorem 5 *A weak separator for a set of n blue points and a set of n red points can be computed in $O(nm \log m + n \log n)$ time, where m is the number of misclassified points.*

We study the following generalization: Let B be a set of n blue bounded convex polygons, and let R be a set of n red bounded convex polygons. Let the total complexity of the polygons in $B \cup R$ be N . A weak separator for B and R is a line l such that the sum of the number of blue polygons that lie strictly below l and the number of red polygons that lie strictly above l is minimum (or vice versa). Given a blue polygon b_i , we dualize it as follows. Let $e_1, \dots, e_j$ be the sequence of edges that bound b_i from above, ordered from right to left. Let $l(e_1), \dots, l(e_j)$ be the sequence of lines extending these edges. The dual of b_i is defined to be the polygonal chain on the points $\mathcal{D}(l(e_1)), \dots, \mathcal{D}(l(e_j))$, to which two half-lines with slope ∞ and $-\infty$ are added, originating at $\mathcal{D}(l(e_1))$ and $\mathcal{D}(l(e_j))$, respectively. The dual $\mathcal{D}(b_i)$ is easily seen to be an unbounded x -monotone polygonal chain that is convex, and has the same complexity as b_i . A red polygon r_i is dualized in an analogous way, only now we take the sequence of edges that bound r_i from below, ordered from left to right.

By duality it easily follows that if the blue polygons are mutually disjoint, then any pair of chains dual to a pair of blue polygons intersects in at most 2 points. Furthermore, if any pair of blue polygons intersects in at most s points, then any pair of chains dual to two blue polygons intersects in at most s points. An algorithm similar to the one presented for the sets of points, combined with Theorem 4, yields:

Theorem 6 *Let B be a set of n blue bounded convex polygons, let R be a set of n red bounded convex polygons, and let N be the total complexity of the*

polygons in $B \cup R$. Let s be the maximum number of times any pair of equal-colored polygons intersect. If $s \geq 3$ is a constant, a weak separator of B and R can be computed in expected time $O((N/m + m \lambda_s(n/m)) \log^2 n + (N + m^2 \lambda_s(n/m)) \log m)$, where m is the number of misclassified polygons. If $s \leq 2$, the bound is $O((N/m + n) \log^2 n + (N + nm) \log m)$.

Remark: We can redefine the weak separation problem such that the objects intersecting the weak separator are also misclassified. We obtain the same results for this problem.

Remark: Consider the following problem: Given a set S of n disjoint line segments in the plane, find a line that has one segment above it and maximizes the number of segments completely below it. With similar techniques, we can solve the problem in $O(f_m(n) \log^2 n \log m) = O(n \sqrt{m} \log^2 n \log m)$ time, where m is the number of segments not below the line. This time we apply the k -level construction algorithm to construct the 2^i -level for $i = 1, 2, \dots, j$, where j is such that $2^{j-1} < m \leq 2^j$. A binary search for m between these bounds completes the algorithm. Details are omitted in this version of the paper.

4 The line transversal problem

The second application of our results concerns maximum line transversals. We begin with the case of vertical line segments, later we generalize to arbitrary line segments and convex polygons.

Let S be a set of n vertical line segments in the plane, the problem is to find a line that intersects the maximum number of them. We reduce this problem to the weak separation problem as follows. For each segment of S , color its upper endpoint blue, and color its lower endpoint red. We observe that a line l does not intersect a segment if and only if its blue endpoint is below l or its red endpoint is above l . Hence, any line that is a weak separator for the blue and red points is also a maximum line transversal for the set S of vertical line segments. We conclude:

Theorem 7 *Given a set S of n vertical line segments in the plane, a maximum line transversal for*

S can be found in $O(nm \log m + n \log n)$ time, where m is the number of segments that is not intersected.

Remark: By duality, the same result holds for finding the point contained in the maximum number of half-planes.

Next, let S be a set of arbitrary line segments in the plane. For any segment $s \in S$, we make a blue unbounded polygon with three sides, and a red unbounded polygon with three sides. The blue polygon of s is bounded by s itself, and two half-lines that start at the endpoints of s , and are directed vertically downward. The red polygon of s is bounded by s and two half-lines that are directed vertically upward. Notice that any two blue polygons, or any two red polygons, intersect in at most three points.

A line l does not intersect a line segment s if and only if either the blue polygon delimited by s lies strictly below l , or the red polygon delimited by l lies strictly above l . Recall that a line l is a weak separator of a set of blue polygons and a set of red polygons if the number of blue polygons lying below l plus the number of red polygons lying above l is minimum. Then the maximum line transversal problem for line segments can be reduced to the weak separation problem for convex polygons. The dual of each of these polygons is an unbounded x -monotone polygonal convex chain. Since any pair of blue (or red) polygons intersect at most three times, and the total complexity of the chains $N \in O(n)$, we obtain:

Theorem 8 *Given a set S of n line segments in the plane, a maximum line transversal for S can be found in $O((nm \log m + n \log^2 n)\alpha(n))$ expected time, where m is the number of segments that is not intersected. For disjoint line segments, the $\alpha(n)$ -factor can be removed.*

It is easy to see that this approach generalizes to finding line transversals of arbitrary convex polygons. Therefore, we omit the description.

5 Conclusion

This paper presented the first optimal algorithm to compute the levels $1, \dots, k$ in arrangements of

lines. The algorithm extends to levels in arrangements of unbounded x -monotone polygonal convex chains.

These results were applied to obtain quality-sensitive algorithms for optimal weak separators and maximum line transversals, i.e., the algorithms are more efficient when a better separator or line transversal exists.

The following questions arise from this paper:

1. Can the bounds for weak separators and maximum line transversals be improved, e.g. by removing the $\log m$ factor?
2. Can the k -level of n lines be computed in, say, $O(n \log n + f_k(n) \log k)$ time? Such a result would follow immediately from an algorithm to maintain the convex hull of a set of n points known beforehand, with logarithmic cost per update.
3. Can the $(\leq k)$ -levels be constructed optimally in 3-dimensional space? Mulmuley gives an $O(nk^2 \log(n/k))$ expected time algorithm, whereas the combinatorial complexity is known to be $O(nk^2)$. For dimensions greater than 3, Mulmuley presents an optimal $O(n^{\lfloor d/2 \rfloor} k^{\lfloor d/2 \rfloor})$ expected time algorithm, see [16].

References

- [1] P.K. Agarwal, M. Sharir, and P. Schor. Sharp upper and lower bounds on the length of general Davenport-Schinzel sequences. *J. Comb. Theory, Ser. A*, 52:228–274, 1989.
- [2] N. Alon and E. Györi. The number of small semispaces of a finite set of points in the plane. *J. Comb. Theory, Ser. A*, 41:154–157, 1986.
- [3] B. Chazelle. On the convex layers of a planar set. *J. IEEE Trans. Inform. Theory*, 31:509–517, 1985.
- [4] B. Chazelle and H. Edelsbrunner. An optimal algorithm for intersecting line segments in the plane. *J. ACM*, 39:1–54, 1992.
- [5] K.L. Clarkson. New applications of random sampling in computational geometry. *Discr. & Comp. Geom.*, 2:195–222, 1987.

- [6] R. Cole, M. Sharir, and C.K. Yap. On k -hulls and related problems. *SIAM J. Comput.*, 16:61–77, 1987.
- [7] H. Edelsbrunner. *Algorithms in Combinatorial Geometry*. Springer-Verlag, 1987.
- [8] H. Edelsbrunner and L.J. Guibas. Topologically sweeping an arrangement. *J. Comp. Sys. Sci.*, 38:165–194, 1989.
- [9] H. Edelsbrunner, H.A. Maurer, F.P. Preparata, A.L. Rosenberg, E. Welzl, and D. Wood. Stabbing line segments. *BIT*, 22:274–281, 1982.
- [10] H. Edelsbrunner and E. Welzl. Constructing belts in two-dimensional arrangements with applications. *SIAM J. Comp.*, 15:271–284, 1986.
- [11] P. Erdős, L. Lovász, A. Simmons, and E. Strauss. Dissection graphs of planar point sets. In J. Srivastava *et al.*, editor, *A Survey of Combinatorial Theory*, pages 139–149. North-Holland, 1973.
- [12] M.E. Houle. Algorithms for weak and wide separation of sets. *Proc. Int. Workshop on Discr. Alg. and Comp.*, pages 61–68, 1989.
- [13] N. Megiddo. Linear-time algorithms for linear programming in R^3 and related problems. *SIAM J. Comp.*, 12:759–776, 1983.
- [14] K. Mulmuley. A fast planar partition algorithm, I. *Proc. 29th IEEE Symp. Found. of Comp Science*, 1988.
- [15] K. Mulmuley. A fast planar partition algorithm, II. *Proc. 5th ACM Symp. on Comp. Geom.*, pages 33–43, 1989.
- [16] K. Mulmuley. On levels in arrangements and Voronoi diagrams. *Discr. & Comp. Geom.*, 6:307–338, 1991.
- [17] J. O’Rourke. An on-line algorithm for fitting straight lines between data ranges. *Comm. ACM*, 24:574–578, 1981.
- [18] M.H. Overmars and J. van Leeuwen. Maintenance of configurations in the plane. *J. Comp. Sys. Sci.*, 23:166–204, 1981.
- [19] J. Pach, W. Steiger, and E. Szemerédi. An upper bound on the number of planar k -sets. In *Proc. 30th IEEE Symp. Found. of Comp. Science*, pages 72–79, 1989.
- [20] M. Sharir. On k -sets in arrangements of curves and surfaces. *Discr. & Comp. Geom.*, 6:593–613, 1991.